
Architecture RAG

Guide Complet avec Figures Explicatives

Plateforme AgentFlow — Projet de Fin d'Etudes

Auteur	Fedi Khala
Encadrement	Welyne Software Engineering
Date	Mai 2026
Sujet	Conception et implementation d'une plateforme d'agents IA metiers basee sur LLM et RAG

Table des Matieres

1. Qu'est-ce que le RAG ?
2. Pipeline RAG Complet (Figure 1)
3. Etape 1 : Extraction de Documents
4. Etape 2 : Chunking — Strategies de Decoupage (Figure 2)
5. Etape 3 : Embeddings — Vectorisation Semantique (Figure 3)
6. Etape 4 : Stockage Vectoriel (pgvector)
7. Etape 5 : Recherche Hybride (Figure 4)
8. Etape 6 : Generation LLM
9. Architecture Multi-Agents (Figure 5)
10. Auto-Tuning et Evaluation Automatique (Figure 6)
11. RAG Entierement Configurable (Figure 7)
12. Console de Test Ameliorée (Figure 8)
13. Synthese et Apport du Projet

1. Qu'est-ce que le RAG ?

Le **RAG (Retrieval-Augmented Generation)** est une architecture hybride qui combine la **recherche d'informations** dans une base de connaissances avec la **generation de texte** par un modele de langage (LLM). Le principe est simple : au lieu de demander au LLM de repondre uniquement depuis sa memoire (ce qui peut produire des hallucinations), on va d'abord chercher les passages pertinents dans les documents de l'entreprise, puis les injecter comme contexte pour que le LLM genere une reponse fondee et verifiable.

Probleme resolu : Un LLM classique ne connait pas les documents internes de votre entreprise. Le RAG lui donne acces a cette connaissance de maniere dynamique, sans necessiter de re-entrainement.

Le pipeline RAG se decompose en **deux phases** distinctes : la **phase d'indexation** (traitement et stockage des documents, executee une seule fois) et la **phase de requete** (recherche et generation, executee a chaque question posee).

2. Pipeline RAG Complet

La figure ci-dessous presente le pipeline RAG complet tel qu'implemente dans AgentFlow. La partie gauche montre la phase d'indexation (traitement des documents), la partie droite montre la phase de requete (reponse aux questions).

Figure 1 — Pipeline RAG Complet

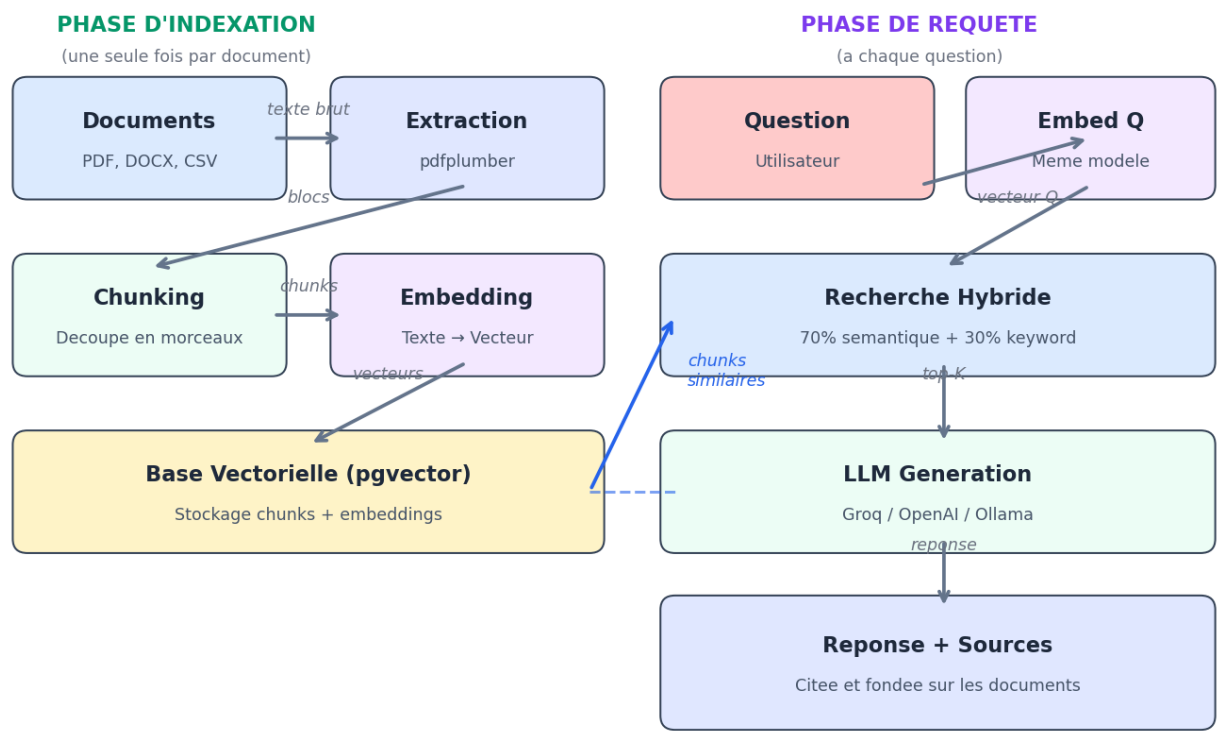


Figure 1 — Pipeline RAG complet : phase d'indexation (gauche) et phase de requete (droite)

La connexion entre les deux phases se fait via la **base vectorielle** (pgvector). Les embeddings stockes durant l'indexation sont interroges durant la requete grace a l'operateur de distance cosinus ($\cos$) de pgvector, qui permet une recherche semantique en $O(\log n)$ grace a l'index IVFFlat.

3. Etape 1 : Extraction de Documents

L'extraction consiste a convertir les fichiers bruts (PDF, DOCX, Excel, CSV, TXT) en texte structure exploitable par le pipeline. Chaque format necessite un processeur dedie.

3.1 Processeurs par format

Format	Outil	Particularites
PDF	pdfplumber + PyPDF2	Detection de tableaux (3 methodes), extraction texte + layout
DOCX	python-docx	Paragraphes, tableaux, hierarchie de titres preservee
CSV	pandas	Chaque ligne = un enregistrement, header prepend a chaque chunk
Excel	openpyxl	Lecture feuille par feuille, gestion multi-onglets
TXT	chardet + open()	Detection automatique de l'encodage
HTML	BeautifulSoup	Nettoyage des balises, extraction du contenu semantique

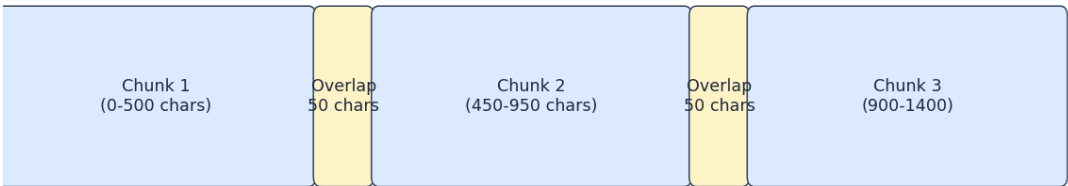
Le resultat de l'extraction est une liste de **blocs de contenu**, chacun ayant un type (texte ou table), un contenu (le texte extrait) et un numero de page. Les tableaux sont convertis en format Markdown pour preserver leur structure.

4. Etape 2 : Chunking — Strategies de Decoupage

Le chunking est l'operation qui decoupe le texte extrait en morceaux (chunks) de taille geree. Chaque chunk doit etre assez grand pour contenir une idee complete, mais assez petit pour etre precis lors de la recherche. AgentFlow supporte trois strategies de chunking :

Figure 2 — Les 3 Strategies de Chunking

Chunking Fixe (RecursiveCharacterTextSplitter)



Chunking Semantique (par sujet)



Chunking Hierarchique (Parent-Enfant)



Figure 2 — Les trois strategies de chunking : fixe, semantique et hierarchique

4.1 Chunking fixe

Le texte est decoupe tous les N caracteres (ex: 500) avec un **chevauchement** (overlap) de M caracteres (ex: 50). L'overlap repete les derniers mots du chunk precedent au debut du suivant, evitant de couper une idee en deux. C'est la methode la plus simple mais elle peut produire des chunks incoherents.

4.2 Chunking semantique

Utilise les embeddings pour detecter les **frontieres de sujet**. On embed chaque phrase, puis on compare les embeddings de phrases consecutives. Quand la similarite chute en dessous d'un seuil, on coupe. Resultat : chaque chunk correspond a un sujet coherent. Les tailles varient mais la coherence est maximale.

4.3 Chunking hiérarchique (parent-enfant)

Crée **deux niveaux** de chunks : des parents larges (~2000 chars, ex: un chapitre entier) et des enfants petits (~500 chars, ex: un article). La recherche cible les enfants (précision), mais le contexte envoyé au LLM inclut le parent (richesse). C'est la stratégie la plus performante car elle combine précision de retrieval et richesse de contexte.

5. Etape 3 : Embeddings — Vectorisation Semantique

L'embedding est la transformation d'un texte en un **vecteur numerique** (une liste de nombres) qui encode le **sens** du texte. Un modele d'embedding (comme BGE-M3) a ete entraine sur des milliards de phrases et a appris que des textes de sens similaire produisent des vecteurs proches dans l'espace mathematique.

Figure 3 — Espace Vectoriel d'Embeddings (representation 2D simplifiee)

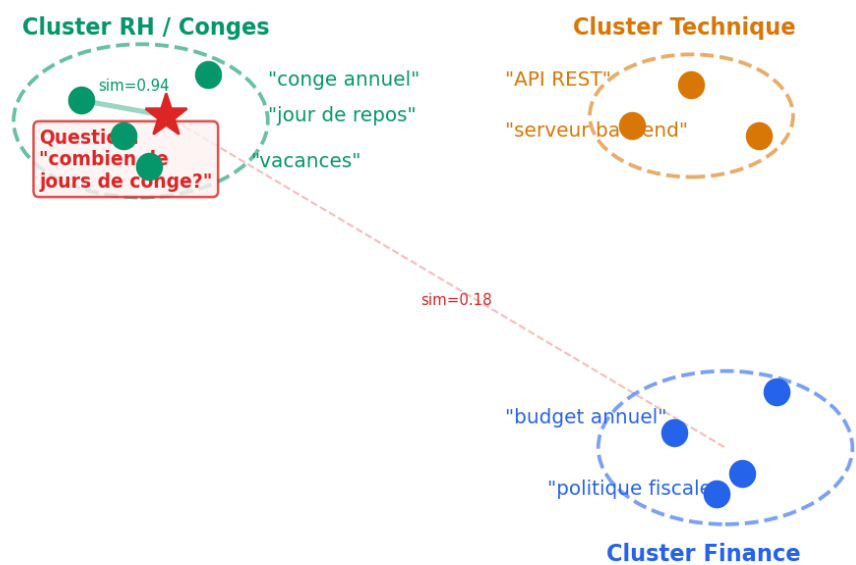


Figure 3 — Espace vectoriel d'embeddings (representation 2D simplifiee)

Dans la figure ci-dessus, les textes parlant de conges (en vert) sont regroupes dans le meme coin de l'espace, tandis que les textes financiers (en bleu) sont dans un coin oppose. Quand l'utilisateur pose une question sur les conges, son vecteur (etoile rouge) atterrit pres du cluster RH — et le systeme retrouve les bons chunks.

5.1 Modeles d'embedding supportes

Modele	Dimensions	Type	Qualite
BGE-M3 (BAAI)	1024	Local / Gratuit	Excellente (multilingue)
BGE-base-en	768	Local / Gratuit	Bonne (anglais)
all-MiniLM-L6-v2	384	Local / Gratuit	Correcte (rapide)
text-embedding-3-small	1536	API OpenAI	Excellente
embed-multilingual-v3	1024	API Cohere	Excellente (multilingue)

5.2 Similarite cosinus

La **similarite cosinus** mesure l'angle entre deux vecteurs. Si deux vecteurs pointent dans la meme direction, le cosinus de l'angle est proche de 1 (textes similaires). S'ils pointent dans des directions opposees, le cosinus est proche de 0 (textes sans rapport). C'est cette mesure que pgvector utilise via l'operateur `<=>`.

6. Etape 4 : Stockage Vectoriel (pgvector)

pgvector est une extension de PostgreSQL qui ajoute un type de donnees **Vector** et des operateurs de distance (cosinus, euclidienne, produit scalaire). Chaque chunk est stocke avec son texte original ET son vecteur d'embedding.

6.1 Pourquoi pgvector plutot qu'une base vectorielle dediee ?

- **Pas de service supplementaire** : pgvector s'integre directement dans PostgreSQL, pas besoin de deployer Pinecone ou Weaviate en plus.
- **SQL standard** : les requetes vectorielles s'ecrivent en SQL, pas un langage proprietare.
- **Transactions ACID** : les chunks et les embeddings beneficient des garanties PostgreSQL.
- **Performance** : avec l'index IVFFlat (ou HNSW), la recherche est en $O(\log n)$ meme avec des millions de vecteurs.

6.2 Index vectoriels

Index	Avantages	Cas d'usage
IVFFlat	Rapide a construire, bon pour < 1M vecteurs	Usage actuel dans AgentFlow
HNSW	Meilleure qualite de recherche, plus lent a construire	Recommande pour > 100K chunks

7. Etape 5 : Recherche Hybride

Quand un utilisateur pose une question, le systeme effectue une **recherche hybride** combinant deux approches complementaires :

- **Recherche semantique (70%)** : la question est transformee en vecteur, pgvector trouve les chunks dont les vecteurs sont les plus proches. Comprend le *sens*, pas les mots exacts.
- **Recherche par mots-cles (30%)** : un scoring TF-IDF basique compte les mots communs entre la question et chaque chunk. Trouve les correspondances *exactes* (codes, noms propres).

Figure 4 — Recherche Hybride : comparaison des scores

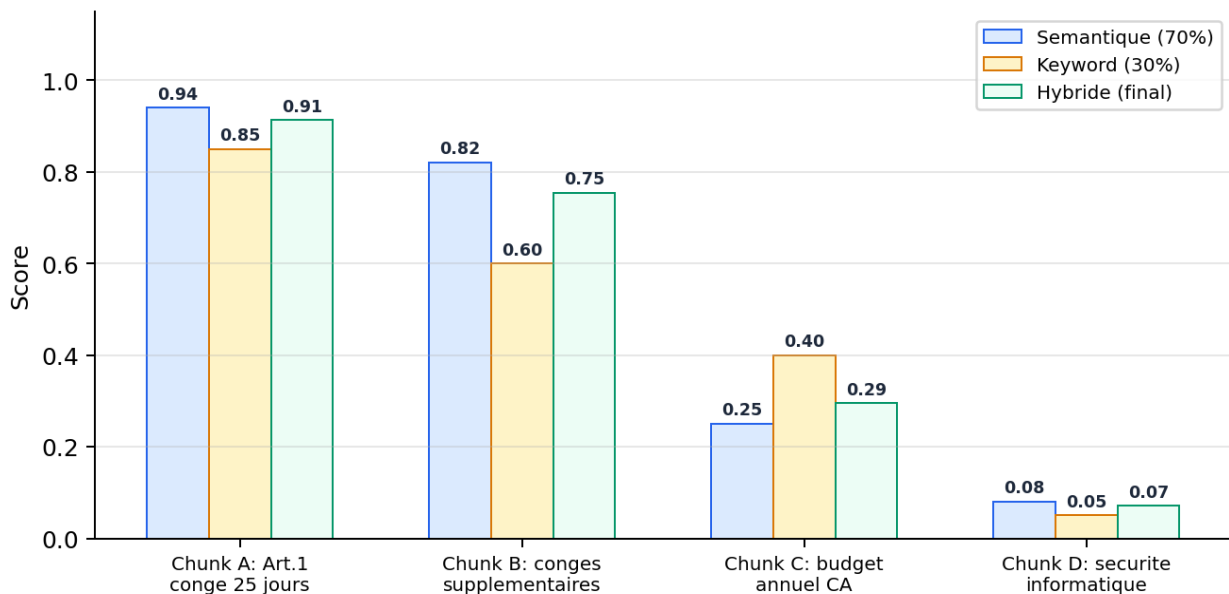


Figure 4 — Recherche hybride : comparaison des scores semantique, keyword et combine

La formule de combinaison est :

$$\text{Score} = 0.70 \times \text{Score_semantique} + 0.30 \times \text{Score_keyword}$$

Ce ratio est configurable par espace RAG dans la prochaine phase du projet. Certains cas d'usage necessitent plus de poids keyword (documents avec des codes specifiques) tandis que d'autres beneficient d'un poids semantique plus eleve (documents conceptuels).

8. Etape 6 : Generation LLM

Les top-K chunks les plus pertinents sont assembles en un **contexte** et envoyes au LLM avec un prompt systeme structure. Le prompt force le LLM a :

- Repondre **uniquement** a partir du contexte fourni (pas d'invention)
- Dire explicitement s'il ne trouve pas l'information
- **Citer les sources** (nom du document, numero de page, score de pertinence)
- Repondre dans la **memme langue** que la question

8.1 Modeles LLM supportes

Provider	Modeles	Type	Latence
Groq	Llama 3.3 70B Versatile	Gratuit	~200ms
OpenAI	GPT-4o, GPT-4o-mini	Payant	~800ms
Ollama	Tout modele auto-heberge	Gratuit (local)	Variable

9. Architecture Multi-Agents

Pour ameliorer la robustesse et la qualite des reponses, le systeme adopte une architecture **multi-agents** ou chaque agent a une responsabilite specifique. Cette separation permet de retenter les etapes defaillantes independamment.

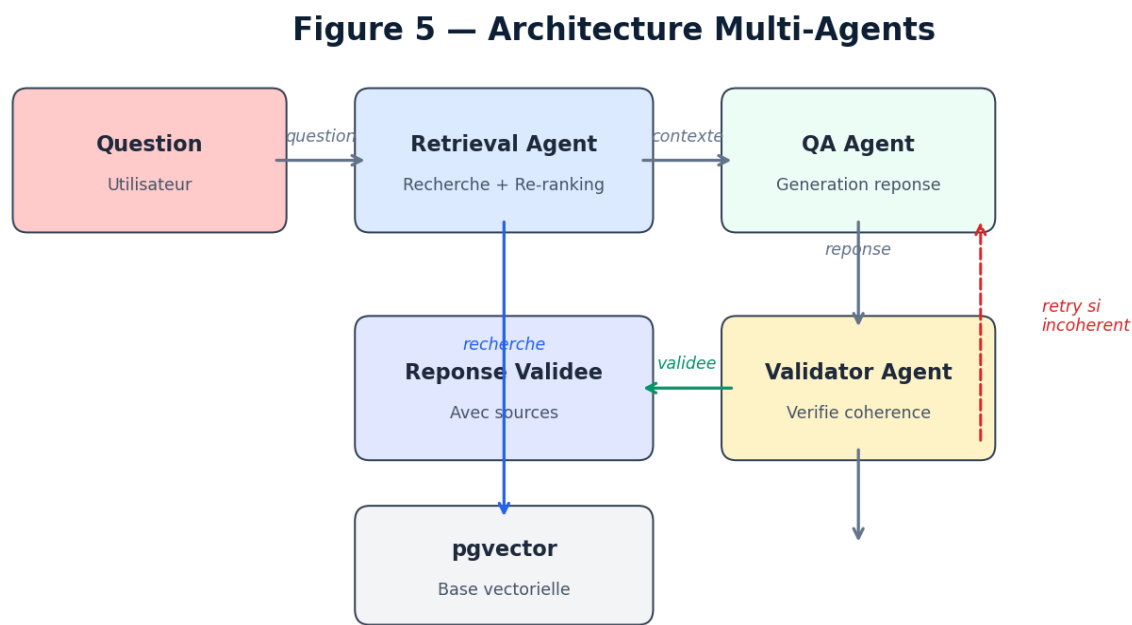


Figure 5 — Architecture multi-agents avec boucle de validation

Agent	Role	Entree	Sortie
Retrieval Agent	Recherche hybride + re-ranking des chunks	Question + vecteur	Top-K chunks ordonnees
QA Agent	Generation de la reponse a partir du contexte	Question + contexte	Reponse brute
Validator Agent	Verifie que la reponse est coherente et fondee	Reponse + contexte	Valide ou retry

10. Auto-Tuning et Evaluation Automatique

L'un des apports majeurs d'AgentFlow est le mecanisme d'**auto-optimisation** des parametres. Au lieu de configurer manuellement `chunk_size`, `top_k` et le modele d'embedding, le systeme teste automatiquement plusieurs combinaisons et selectionne la meilleure.

Figure 6 — Auto-Tuning et Evaluation Automatique

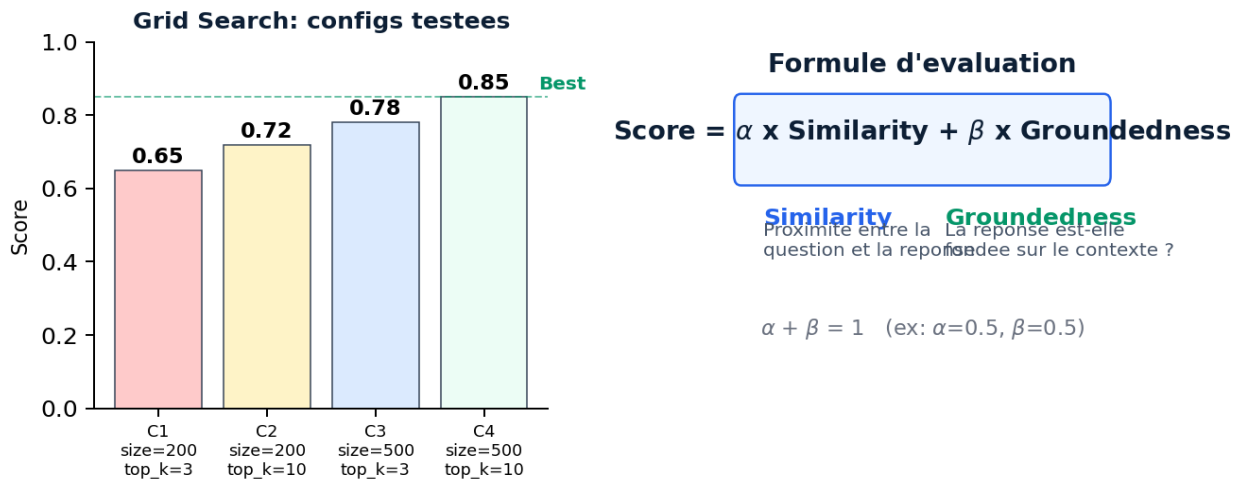


Figure 6 — Auto-tuning par grid search et formule d'evaluation automatique

Le score d'evaluation combine deux metriques :

- **Similarity** : mesure la proximite semantique entre la question et la reponse generee. Un score eleve signifie que la reponse est pertinente par rapport a la question posee.
- **Groundedness** : mesure dans quelle mesure la reponse est supportee par le contexte fourni. Un score eleve signifie que le LLM n'a pas hallucine — sa reponse est fondee sur les documents.

11. RAG Entierement Configurable

L'objectif de la prochaine phase est de rendre **chaque composant du pipeline configurable** par espace RAG. Le developpeur IT choisit, depuis l'interface, le format d'upload, la strategie de chunking, le modele d'embedding, le LLM, les parametres de recherche et le prompt systeme.

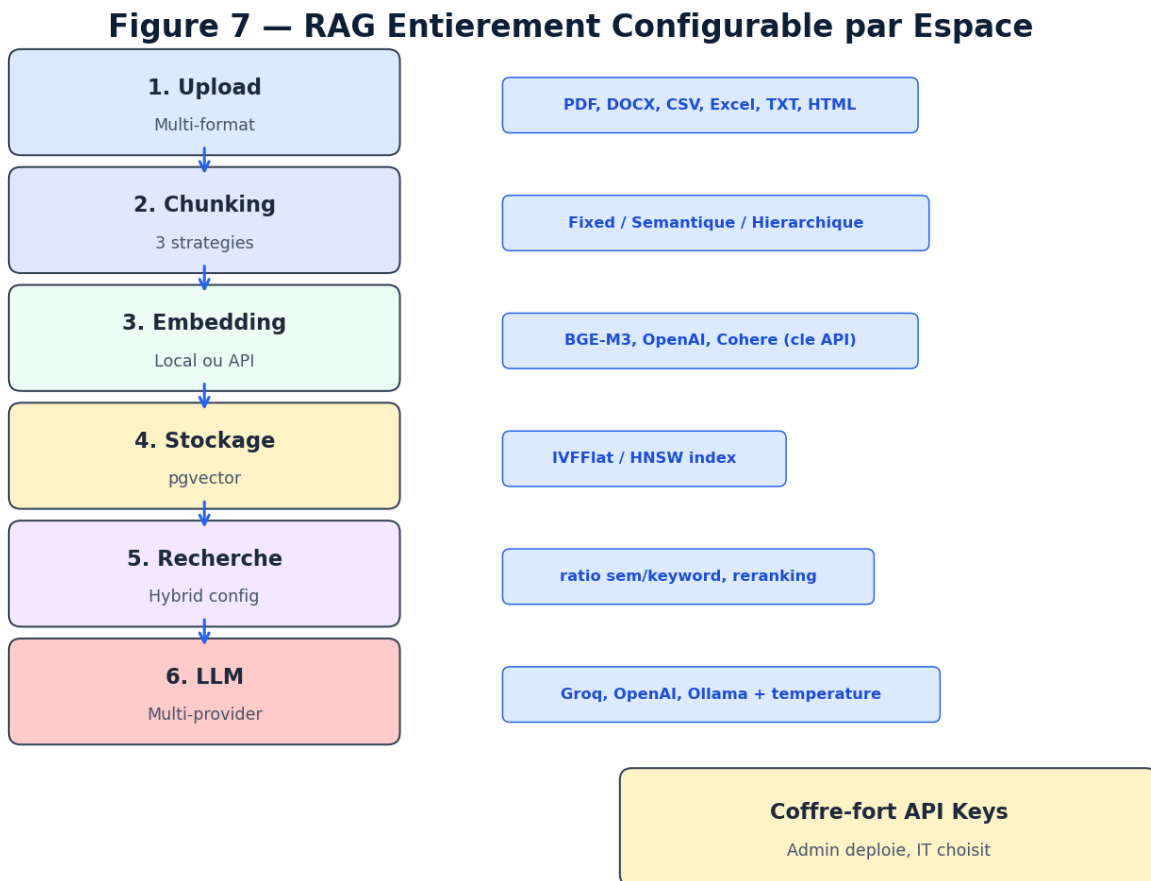


Figure 7 — Pipeline RAG entierement configurable : chaque etape est parametrable

11.1 Gestion des cles API

L'Admin deploie les cles API (OpenAI, Cohere, Groq, HuggingFace) au niveau de l'organisation. Les cles sont chiffrees avec **Fernet** (chiffrement symetrique). Le developpeur IT peut aussi fournir sa propre cle API par espace RAG. Les modeles locaux (BGE-M3, Llama via Ollama) restent disponibles sans cle.

12. Console de Test Ameliorree

La console de test offre une visibilite complete sur le fonctionnement interne du pipeline, permettant au developpeur IT d'optimiser la configuration de chaque espace RAG.

Figure 8 — Console de Test Ameliorree

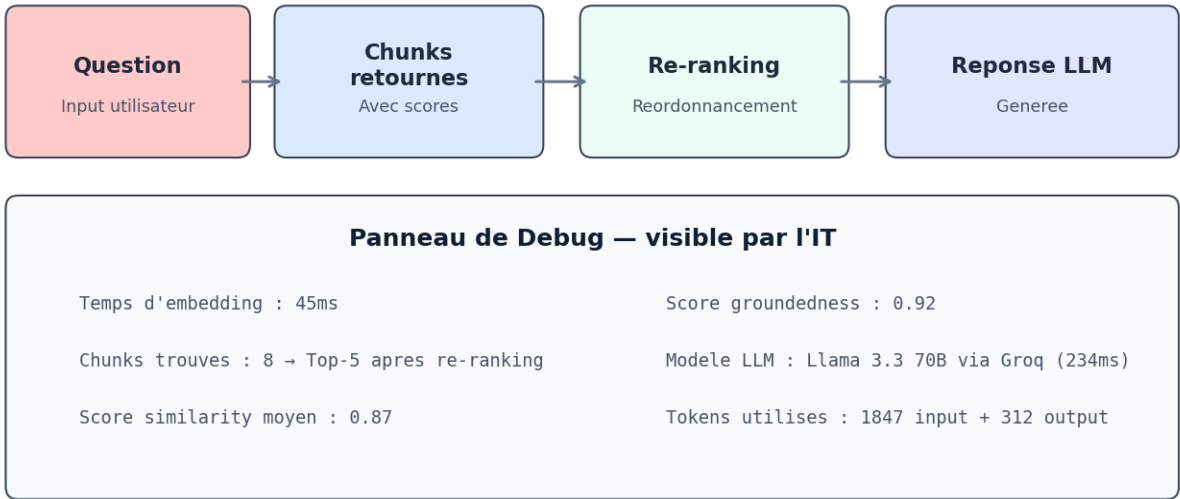


Figure 8 — Console de test avec panneau de debug pour le developpeur IT

Le panneau de debug affiche en temps reel : le temps d'embedding, le nombre de chunks trouves et retenus apres re-ranking, les scores de similarite et de groundedness, le modele LLM utilise avec son temps de reponse, et le nombre de tokens consommes.

13. Synthèse et Apport du Projet

Ce document a presente l'architecture RAG complete d'AgentFlow, de l'extraction de documents jusqu'a la generation de reponses par LLM. Les 8 figures illustrent chaque etape du pipeline et chaque mecanisme avance (chunking semantique, espace vectoriel, recherche hybride, multi-agents, auto-tuning).

13.1 Apports principaux du projet

Apport	Description
Pipeline RAG complet	6 etapes couvrant extraction, chunking, embedding, stockage, recherche et generation
3 strategies de chunking	Fixe, semantique et hierarchique — configurables par espace
Recherche hybride	Combinaison semantique + keyword avec ratio ajustable
Architecture multi-agents	Retrieval, QA et Validator agents avec boucle de retry
Auto-tuning	Optimisation automatique des parametres par grid search
Evaluation automatique	Score = Similarity + Groundedness pour mesurer la qualite
RAG configurable	Chaque composant parametrable par espace depuis l'interface
Coffre-fort API Keys	Cles chiffrees Fernet, support modeles locaux et payants

Ce projet depasse le cadre d'un simple outil technique pour devenir une veritable plateforme d'ingenierie des systemes RAG, avec un fort potentiel d'extension et d'application dans des contextes professionnels.